

Autonomous Vehicle System

Hussein Mohamud, Aidan Elkins, Emily Bailey, William Varda, Rami
Ibraheem

System Description:

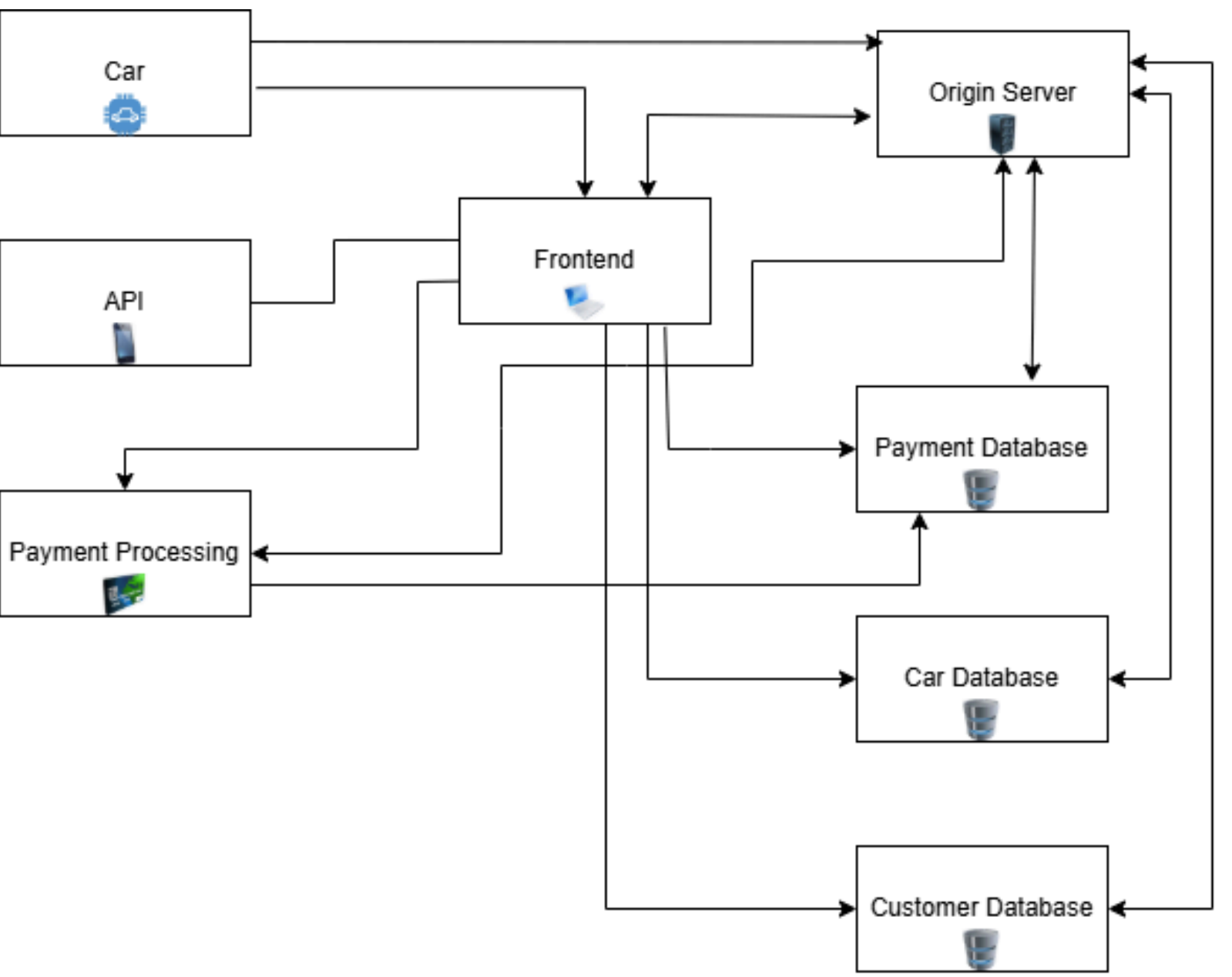
This system handles car rentals and overall day-to-day management for BeAvis. It has the option of being accessible via both website and mobile application (with latest IOS or Android version), each with the same utilities. Users can enable GPS to view nearby locations as well as their direction once a rental is requested.

Users must create an account with their email and a password to make rentals. After verifying an email, their account will be created and they will then have the option to enable two-factor authentication with a separate authenticator app at any time, although it is not mandatory. Users can make rentals, view rental history, find locations, and link their method of payment. Payment information is secure and in a different location from other data in case of breach.

Rental agreement contracts are also given and signed here; the contracts will be kept secure along with customer signatures.

Employees can log into the system and are able to access customers' rental status and contracts, see cars available for rental, and update car status.

Software Architecture Overview:



SWA Description:

Component name: **Car**

Purpose: Processes requests for Car information, communicating with Car database

Connects to: Frontend, Origin Server

Component name: **API**

Purpose: Calls external APIs for routing requests to back end servers

Connects to: Frontend

Component name: **Payment Processing**

Purpose: Processes payments made, as well as calls external payment systems

Connects to: Payment Database

Component name: **Frontend**

Purpose: User interface, client side requests for information and updates

Connects to: Origin Server, Payment Database, Car Database, Customer Database, Payment Processing, API

Component name: **Origin Server**

Purpose: Coordinates cross database communication, store and update information

Connects to: Frontend, Payment Database, Car Database, Customer Database, Payment Processing

Component name: **Payment Database**

Purpose: Database isolation protects user information and stores payment data

Connects to: Frontend, Origin Server, Payment Processing

Component name: **Car Database**

Purpose: Stores total car information

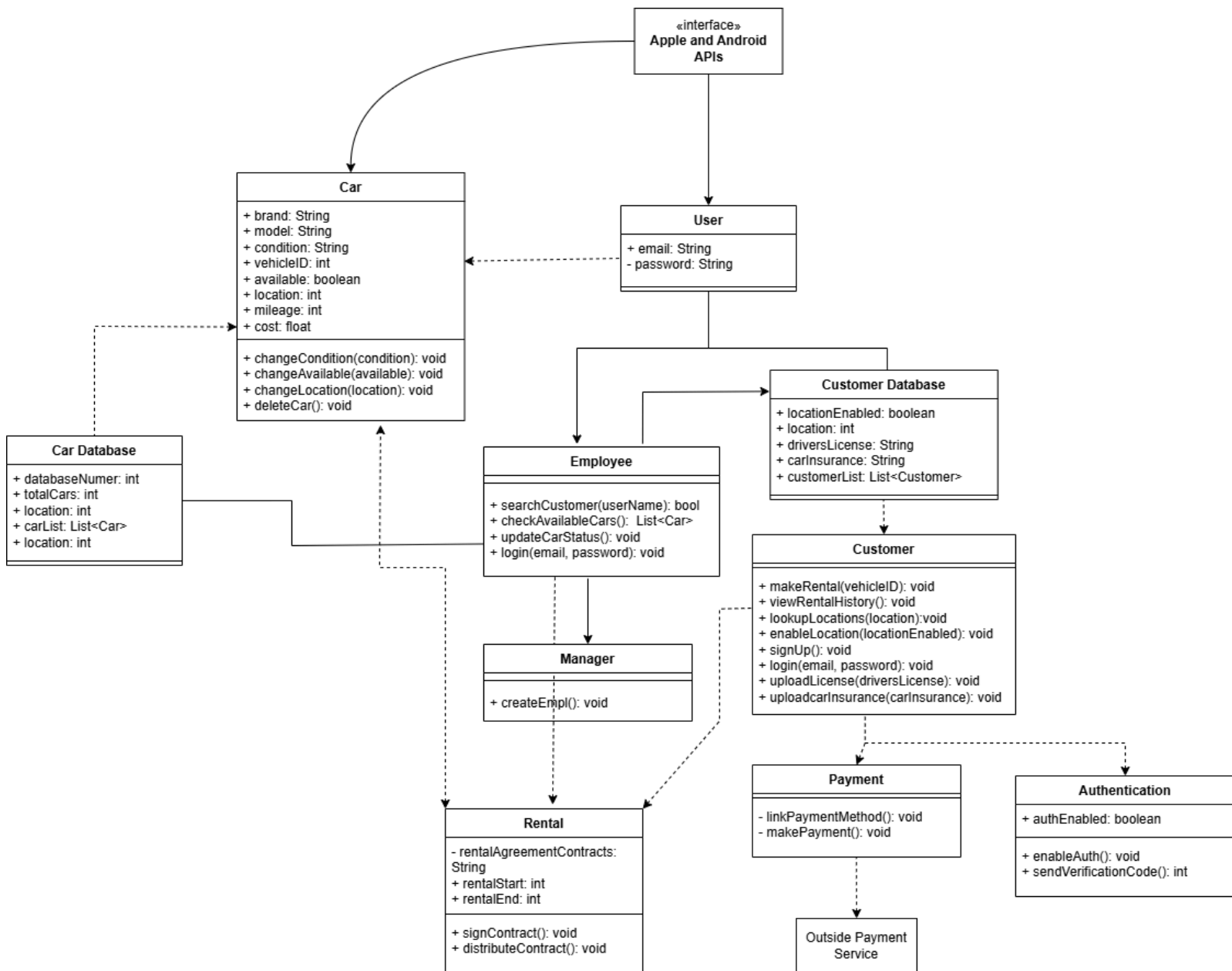
Connects to: Frontend, Origin Server

Component name: **Customer Database**

Purpose: Database isolation protects user information and store user data

Connects to: Frontend, Origin Server

UML Diagram



Description of Classes, Associated Attributes & Operations

Class name: **Car Database**

Purpose: Abstract class to store and update car object within lists

Parameters

- + databaseNumer: int
- + totalCars: int
- + location: int
- + carList: List<Car>
- + location: int

No Methods

Class name: **Car**

Purpose: To create and update all stored car objects

Parameters

- + brand: String
- + model: String
- + condition: String
- + vehicleID: int
- + available: boolean
- + location: int
- + mileage: int
- + cost: float

Methods

- + changeCondition(condition): void
updates condition for vehicle
 - + changeAvailable(available): void
updates availability for vehicle
 - + changeLocation(location): void
updates location for vehicle
 - + deleteCar(): void
removes car from list
-

Class name: **User**

Purpose: Abstract class for both user types

Parameters

- + email: String

- password: String

No Methods

Class name: **Employee**

Purpose: Inherits from, contains employee objects and associated roles

Parameters

Methods

- + searchCustomer(userName): bool
finds customer in within list, returns true if found
 - + checkAvailableCars(): List<Car>
searches list for cars with list, returns list of available
 - + updateCarStatus(): void
updates car availability status
 - + login(email, password): void
logs employees in to access workplace methods
-

Class name: **Manager**

Purpose: Inherits from Employee

No Parameters

Methods

- + createEmpl(): void
creates new employee object with sign in capabilities
-

Class name: **Customer Database**

Purpose: Inherited from User, abstract class to create and update all stored customer objects

Parameters

- + locationEnabled: boolean
- + location: int
- + driversLicense: String
- + carInsurance: String
- + customerList: List<Customer>

No Methods

Class name: **Customer**

Purpose:

No Parameters

Methods

- + makeRental(vehicleID): void
start rental process with vin
 - + viewRentalHistory(): void
connects to user's prior rental history
 - + lookupLocations(location): void
searches car list to find cars based on location
 - + enableLocation(locationEnabled): void
alters location boolean true for on false for off
 - + signUp(): void
initiates creation of new customer object - new account creation with input checks to verify valid email and password (calls on sendVerificationCode() method in Authentication class)
 - + login(email, password): void
searches customer database for current customer login info
 - + uploadLicense(driversLicense): void
uploads string to verify rental requirement for drivers license
 - + uploadcarInsurance(carInsurance): void
uploads string to verify rental requirement for car insurance
-

Class name: **Rental**

Purpose: Tracks active rental contracts (private to maintain secure information) to complete signing, processing, and distributing

Parameters

- rentalAgreementContracts: String
- + rentalStart: int
- + rentalEnd: int

Methods

- + signContract(): void
completes virtual e-signature request
 - + distributeContract(): void
signed contract distributed
-

Class name: **Payment**

Purpose: Secured by separate class and private methods, class functions to process payments and handle payment information

No Parameters

Methods

- linkPaymentMethod(): void
allows user to link their payment method to their account for purchases
 - makePayment(): void
allows for users to make the payment. Also contacts external payment service
-

Class name: **Authentication**

Purpose: Allows two factor authentication optional service and runs email authentication for new account creation

Parameters

- + authEnabled: boolean

Methods

- + enableAuth(): void
allows for the user to enable 2 factor authentication
 - + sendVerificationCode(): int
sends the user's email a verification code
-

Development plan and timeline

Partitioning of tasks:

Rami: Documentation & Software Summary - assist in UML

Hussein: Primary UML

Aidan: Software Architectural Diagram - assist in UML

William: Primary UML

Emily: Class and Method Descriptions - assist in UML

Team member responsibilities:

Submission of final project: Emily

Verify Git Organization: Aidan

Assist with Git Organization: Hussein

Finalizing UML: William & Rami